

Analytic_t_dep_sim_mem_saver.py

A complete walkthrough of the time-dependent ULDM stellar-heating simulation

Physics · algorithms · data layout · memory engineering

Reference document — written to be read end-to-end once,
then dipped into, and finally used as a rewrite specification.

Contents

How to read this document	4
Part I — The physics being simulated	5
1.1 The ULDM halo as a sum of eigenstates	5
1.2 Density	5
1.3 The static (diagonal) density	5
1.4 From density to potential: the multipole Poisson solve	6
1.5 From potential to acceleration	6
1.6 Stars are test particles	7
1.7 Time integration	7
Part II — Data layout: the four index spaces	8
2.1 The indices	8
2.2 The mapping arrays	8
2.3 The central object: $\psi_{lm}(r)$	8
2.4 Bandwidth: L , $L_{\text{max_out}}$, and the truncation	9
2.5 L-sharding alignment	10
Part III — Setup: <code>__init__</code> and initialising simulation	11
3.1 <code>StellarSimTDep.__init__</code>	11
3.2 <code>initialising_simulation</code> — the ten stages	11
Stage 1 — the eigenstate library and its cache	12
Stage 3 — <code>dtype</code> , padding, sharding of <code>R_j_r</code>	12
Stage 5 — the amplitudes	13
Stage 8 — initial conditions	13
Stage 9 — the force callback	13
Stage 10 — the timestep	14
Part IV — <code>Simulation_Particle</code>	15
4.1 The velocity-dispersion definition	15
Part V — Building the density field	16
5.1 Call graph	16
5.2 <code>MSS.sparse_a_u_j_matmul</code> — the hot kernel	16
Step 1 — <code>precompute_bin_blocks</code> (host, numpy, once)	16
Step 2 — <code>_sort_modes_for_sphht</code> (host, once)	17
Step 3 — the run-time scan	17
Hoisting <code>aj_phase</code> out of the <code>r-chunk</code> loop	18
Choosing <code>sparse_k_batch</code>	18
5.3 The SphHT round trip	18

5.4 The Gaunt path (SphHT = False)	19
5.5 The ramp	19
Part VI — The per-star Poisson solve	21
6.1 insert_particle_rho_lm_and_get_philm — grid refinement, not mass insertion	21
6.2 PS.compute_phi_lm_and_deriv	21
Ratio folding — the precision fix	21
l-banding — the launch-overhead fix	22
6.3 combine_acc and the angular functions	22
Part VII — The main loop and checkpointing	24
7.1 run_simulation	24
7.2 time_step_particle	24
7.3 The frozen-phase approximation	25
7.4 Checkpointing	25
Part VIII — Memory and performance engineering	26
8.1 The mixed-precision contract	26
8.2 Sharding	26
8.3 The bound-method JIT constant trap	26
8.4 The chunking knobs, summarised	27
8.5 Determinism	28
Part IX — A guide to rewriting this	29
9.1 Suggested module structure	29
9.2 Build order, with a test at each stage	29
9.3 Traps, ranked	30
9.4 Things worth changing	30
9.5 Shape reference	30
9.6 Closing note	31

How to read this document

This document explains, in full, what `Analytic_t_dep_sim_mem_saver.py` does and why every non-obvious line is the way it is. It is written for one specific purpose: so that you can throw the file away and write it again from scratch, correctly, without rediscovering the traps.

The code is a *stellar-heating* experiment. A population of massless test stars orbits inside a fuzzy-dark-matter (ULDM / scalar-field dark matter) halo whose density field is not smooth: it is the modulus-squared of a coherent superposition of energy eigenstates, and therefore fluctuates on the de Broglie scale and on the eigenstate beat timescale. Those fluctuations scatter the stars, and the observable is the resulting diffusion in orbital radius and velocity dispersion over ~ 10 Gyr.

Almost all the complexity in the file is not physics. It is the fact that the quantities involved are enormous — tens of millions of modes, arrays that would be tens of gigabytes if written naively — and that everything has to be expressed as fixed-shape JAX operations that compile once and then run twelve thousand times. So this document is organised in three passes:

1. **The physics** (Part I). What is actually being computed. Read once.
2. **The code** (Parts II–VI). Every function, in dependency order, with the shapes and the index conventions that make the shapes make sense.
3. **The engineering** (Parts VII–VIII). Why the code is contorted: dtypes, sharding, chunking, and the JIT constant-folding trap. This is where the real knowledge is, and it is the part you would get wrong on a rewrite.

Part IX is a rewrite guide: the order in which to build it, what to test at each stage, and the list of mistakes that are easy to make and hard to detect.

Conventions used throughout. Array shapes are given as (A, B) . Index letters are fixed and used consistently: r for radial grid bin, j for a radial eigenstate (n, l) , k for a full mode (n, l, m) , u for a unique angular pair (l, m) , p for a star. These four index letters are the single most important thing in the file; Section 3 defines them precisely and everything afterwards depends on them.

Part I — The physics being simulated

1.1 The ULDM halo as a sum of eigenstates

Ultra-light dark matter is modelled as a single classical scalar field ψ obeying the Schrödinger-Poisson system. The halo is taken to be in a quasi-stationary state, so ψ is expanded in the energy eigenbasis of a *fixed* background gravitational potential (a core-NFW-with-tides fit, supplied by the `jaxsp` library). Each eigenstate is separable:

$$\psi_{nlm}(r, \theta, \phi, t) = R_{nl}(r) \cdot Y_{lm}(\theta, \phi) \cdot \exp(-i E_{nl} t)$$

The full field is a coherent superposition over all bound eigenstates, with complex amplitudes a :

$$\psi(r, \theta, \phi, t) = \sum_{nlm} a_{nlm} \cdot R_{nl}(r) \cdot Y_{lm}(\theta, \phi) \cdot \exp(-i E_{nl} t)$$

The eigenstates are indexed by principal number n , angular momentum l , and azimuthal number m . The radial function R_{nl} and energy E_{nl} do not depend on m — this degeneracy is the origin of the j / k index split that dominates the code (Section 3).

The amplitudes are set stochastically. Their *moduli* are fixed by requiring that the time-averaged density reproduce the target halo profile — `jaxsp` returns these as `aj_2`, i.e. $|a|^2$ per eigenstate — while their *phases* are drawn uniformly on $[0, 2\pi)$. This is the standard Gaussian-random-field construction of a virialised ULDM halo: the halo is a specific random realisation, not an ensemble average.

In the code the random phase is drawn **per k-mode**, i.e. independently for each (n, l, m) , using a fixed `PRNGKey(42)`. The modulus `sqrt(aj_2[j])` is shared by all m belonging to the same (n, l) . So $|a_{nlm}|$ is m -independent but `arg(a_nlm)` is not — that is exactly the isotropy assumption that makes the static density spherically symmetric (Section 1.3).

1.2 Density

The mass density is the modulus-squared of the field, scaled to the halo mass:

$$\rho(r, \theta, \phi, t) = M_{\text{tot}} \cdot |\psi(r, \theta, \phi, t)|^2$$

Expanding the square produces diagonal terms ($j = j'$) and off-diagonal cross terms ($j \neq j'$). The diagonal terms are time-independent. The off-diagonal terms carry `exp(-i(E_j - E_j')t)` and are the entire source of the time dependence — the "interference", the granularity, the thing that heats the stars.

The code never expands this sum. It exploits the fact that squaring is trivial on a real-space grid, and expensive in harmonic space, and so does a round trip. That round trip is the `SphHT` path (Section 5.3).

1.3 The static (diagonal) density

Averaging ρ over time kills every off-diagonal term, leaving only $j = j'$. Because the amplitude moduli are m -independent, the sum over m can be collapsed with the spherical-harmonic addition theorem,

$$\sum_m |Y_{lm}(\theta, \phi)|^2 = (2l + 1) / 4\pi$$

which is a constant. The time-averaged density is therefore exactly spherically symmetric:

$$\rho_{\text{static}}(r) = M_{\text{tot}} \cdot \sum_j w_j \cdot |R_j(r)|^2$$

$$w_j = |a_j|^2 \cdot (2l_j + 1) / 4\pi$$

This is the smooth halo profile the simulation is nominally built around. It is used for three separate things, and it is worth being clear that they are different uses of the same object:

- **Initial conditions.** The enclosed mass $M_{\text{enc}}(r) = \int 4\pi r'^2 \rho_{\text{static}} dr'$ gives a circular velocity $v_{\text{circ}} = \sqrt{G M_{\text{enc}}(r) / r}$, used to place stars on circular orbits.
- **The ramp baseline.** At $t = 0$ the stars see ρ_{static} exactly; the fluctuating part is switched on linearly over `ramp_time` (Section 5.5). Without this the stars are hit with the full granular field instantaneously and the initial transient contaminates the measured heating rate.
- **A diagnostic.** ρ_{static} is spherically symmetric, so its only nonzero harmonic coefficient is ($l=0, m=0$), with value $\rho_{\text{static}}(r) \cdot \sqrt{4\pi}$ since $Y_{00} = 1/\sqrt{4\pi}$.

The last point is a genuine computational saving, not just a remark. Evaluating ρ_{static} at an arbitrary radius needs no spherical transform at all — it is one dot product against w_j . That is what `compute_rho_lm_at_particles_diagonal_only` does.

1.4 From density to potential: the multipole Poisson solve

Stars respond to the gravitational potential Φ satisfying $\nabla^2 \Phi = 4\pi G \rho$. Expanding both in spherical harmonics decouples the equation mode by mode, and the standard Green's-function solution for each (l, m) is:

$$\Phi_{lm}(r) = -(4\pi G / (2l+1)) \cdot \left[r^{-(l+1)} \int_0^r \rho_{lm}(r') r'^{(l+2)} dr' + r^l \int_r^\infty \rho_{lm}(r') r'^{(1-l)} dr' \right]$$

The first term is the *interior* contribution (mass inside the star's radius), the second the *exterior*. Differentiating with respect to r , the two terms that involve differentiating the integration limits cancel exactly — both equal $\rho_{lm}(r) \cdot r$ — and what survives is remarkably clean:

$$d\Phi_{lm}/dr = -(4\pi G / (2l+1)) \cdot \left[(l/r) \cdot I_{\text{ext}}(r) - ((l+1)/r) \cdot I_{\text{int}}(r) \right]$$

where I_{int} and I_{ext} are exactly the two bracketed integrals above. This is why the code computes each integral once and reuses it for both Φ and $d\Phi/dr$: they differ only by the per- l scalars l/r and $(l+1)/r$. See `Poisson_solver.compute_phi_lm_and_deriv`, Section 6.

1.5 From potential to acceleration

The acceleration is $\mathbf{a} = -\nabla\Phi$, in spherical components:

$$\begin{aligned} a_r &= - \sum_{lm} (d\Phi_{lm}/dr) \cdot Y_{lm}(\theta, \phi) \\ a_\theta &= - (1/r) \cdot \sum_{lm} \Phi_{lm} \cdot \partial Y_{lm}/\partial\theta \\ a_\phi &= - (1/(r \sin\theta)) \cdot \sum_{lm} \Phi_{lm} \cdot \partial Y_{lm}/\partial\phi \end{aligned}$$

Because $Y_{lm} \propto \exp(im\phi)$, the azimuthal derivative is free: $\partial Y_{lm}/\partial\phi = i m Y_{lm}$. The colatitude derivative $\partial Y_{lm}/\partial\theta$ is obtained by forward-mode automatic differentiation (a JVP) through the Y_{lm} evaluation, which costs one extra pass rather than a second transform. See `MSS.compute_Ylm_and_dtheta_jit`.

The sums run over every (l, m) with $l < L_{\text{max_out}}$ — that is $L_{\text{max_out}}^2$ terms, 57 600 of them at $L_{\text{max_out}} = 240$.

1.6 Stars are test particles

The stars have zero mass. In `initialising_simulation` they are added to rebound with `m=0.0`. They do not source the potential, they do not attract each other, and they do not back-react on the ULDM field. The simulation is strictly one-way: the field pushes the stars.

This matters because the method named `insert_particle_rho_lm_and_get_phi_lm` sounds like it inserts a particle's mass into the density. It does not. What it inserts is the star's *radius* as an extra node in the radial integration grid, so that the boundary between the interior and exterior integrals falls exactly on `r_p` rather than being interpolated between grid bins. It is a grid-refinement trick, and nothing more. Section 6.1 walks through it in detail; the name is misleading and you should rename it on a rewrite.

1.7 Time integration

Stars are advanced with `rebound` (IAS15 by default, optionally leapfrog). Rebound owns the orbit integration; the ULDM force enters through rebound's `additional_forces` callback, which is called several times per macro timestep at trial positions chosen by the integrator.

The macro timestep `dt` is chosen as the smaller of two constraints, each divided by a safety factor `dt_override`:

- `T_orb`: the minimum orbital period over the star population, $2\pi r / v_{\text{circ}}$.
- `T_c`: the shortest coherence / beat timescale of the surviving eigenstates, taken as $\min(-1/E_j)$ over the eigenstates that survive the mode cut. (Bound states have $E < 0$, so $-1/E > 0$.)

The ULDM field is frozen within a macro timestep. `Build_rho_lms_for_timestep` evaluates the phase at `t = time_step * dt` and stores it on `self.current_phase`. Every IAS15 force evaluation inside that macro step then uses the same phase. So the potential is piecewise-constant in time, with steps of size `dt`. This is a real approximation and it is why `dt` must resolve `T_c`, not just `T_orb`. It is not documented anywhere in the source. See Section 8.4.

Part II — Data layout: the four index spaces

Nothing in this file makes sense until the index conventions do. There are four index spaces and the code moves between them constantly. Get these wrong on a rewrite and you will produce plausible-looking garbage.

2.1 The indices

Index	Ranges over	Size	Meaning
j	radial eigenstates (n, ℓ)	N_j	A radial function $R_j(r)$ and an energy E_j . Carries an ℓ (as <code>self.l[j]</code>) but no m . This is what <code>jaxsp</code> returns.
k	full modes (n, ℓ, m)	N_{modes_k}	One per (j, m) pair with $-\ell_j \leq m \leq \ell_j$. Each j spawns $2\ell_j + 1$ of them. Carries the complex amplitude $a_j[k]$.
u	unique angular pairs (ℓ, m)	N_{unique}	A destination bin. Many k map to the same u — all the different radial states n sharing that (ℓ, m) .
p	stars	$N_{\text{particles}}$	Test particles.
r	radial grid bins	N_r	Log-spaced from <code>rmin</code> to <code>rmax</code> .

2.2 The mapping arrays

`MSS.precompute_lm_pairs(l)` builds these. It walks every j , and for each one emits $2\ell_j + 1$ k -modes:

```
for j_idx, ell in enumerate(l):
    for m in range(-ell, ell + 1):
        l_for_kmode.append(ell)          # -> lm_l_per_mode[k]
        m_for_kmode.append(m)            # -> lm_m_per_mode[k]
        ind_for_jmode_over_all_k.append(j_idx)  # -> parent_j[k]
        lm_pairs_dict[(ell, m)] += 1
```

Array	Shape	What it holds
<code>parent_j</code>	$(N_{\text{modes}_k},)$	$k \rightarrow j$. Which radial eigenstate mode k belongs to.
<code>lm_idx_per_mode</code>	$(N_{\text{modes}_k},)$	$k \rightarrow u$. Which angular bin mode k scatters into.
<code>lm_l_per_mode</code> , <code>lm_m_per_mode</code>	$(N_{\text{modes}_k},)$	$k \rightarrow \ell, k \rightarrow m$.
<code>lm_pairs</code>	$(N_{\text{unique}}, 2)$	$u \rightarrow (\ell, m)$. The inverse of <code>lm_idx_per_mode</code> .
<code>aj</code>	$(N_{\text{modes}_k},)$ complex	Amplitude per k -mode: $\sqrt{a_{j,2}[\text{parent_j}[k]]} \cdot e^{i\phi_k}$.
<code>self.l</code>	$(N_j,)$	$j \rightarrow \ell$. Comes straight from <code>jaxsp</code> .
<code>eigen_energies</code>	$(N_j,)$ float64	$j \rightarrow E_j$.
<code>R_j_r_fixed</code>	(N_r, N_j)	The radial basis on the background grid.

2.3 The central object: $\psi_{lm}(r)$

Everything funnels through one quantity. Define the *harmonic coefficient of the wavefunction* at radius r :

$$S[u, r] \equiv \psi_{lm}(r, t) = \sum_{\{k : \text{lm_idx}[k] = u\}} a_k \cdot e^{-i E_{\{j(k)\}} t} \cdot R_{\{j(k)\}}(r)$$

This is a **sparse matrix-matrix product**. If you wrote it densely you would build `a_u_j` of shape (N_{unique}, N_j) — a one-hot scatter with N_{modes_k} nonzeros — and compute `a_u_j @ (phase * R.T)`.

The dense matrix scales as roughly $m22^5$ and is >99% zeros. The code never builds it on the SphHT path. Instead S is computed directly from the triplet $(aj, \text{parent}_j, \text{lm_idx})$ by `MSS.sparse_a_u_j_matmul`, which is the single hottest kernel in the program and is dissected in Section 5.2.

The same kernel is called with R of two different shapes and the code is written to be agnostic: $R = R_{j_r_fixed}$ gives (N_r, N_j) and you get ψ_{lm} on the background grid; $R = R_{j_at_parts}$ gives $(N_{particles}, N_j)$ and you get ψ_{lm} at each star's exact radius. In the kernel this leading axis is just called M .

2.4 Bandwidth: L , L_{max_out} , and the truncation

`jaxsp` returns eigenstates up to some l_{max} ; $L = l_{max} + 1$ is the bandwidth of ψ . Because $\rho = |\psi|^2$, squaring two band- L expansions produces angular content up to $l = 2(L-1)$, so the *lossless* bandwidth for ρ is $2L - 1$. That is the default L_{max_out} .

At $m22 = 50$ this is catastrophic: $l_{max} = 1207$, so $L = 1208$ and $L_{max_out} = 2415$. Every $(N_r, L_{max_out}, 2 \cdot L_{max_out} - 1)$ array — and ρ_{lms} is one — would be enormous. L_{out_frac} scales L_{max_out} down, trading aliasing of the high- l density modes for memory.

```
L_max_out_full = 2 * L - 1
if self.SphHT and self.L_out_frac < 1.0:
    #L_sht = max(int(round(self.L_out_frac * L_max_out_full)), L) # <- disabled floor
    L_sht = int(round(self.L_out_frac * L_max_out_full))
    self.L_max_out = L_sht
else:
    self.L_max_out = L_max_out_full
```

Note the commented-out `max(..., L)`. With the floor active, $L_{max_out} \geq L$, so the input ψ modes always fit in the output buffer. With it disabled — the current state — L_{max_out} can fall *below* L , and then some ψ modes have $l \geq L_{max_out}$ and would scatter to out-of-range indices. JAX clamps out-of-range scatter indices silently, which would pile every high- l mode onto the last row of the buffer and quietly corrupt it.

The block guarded by `if self.SphHT and self.L_max_out < L`: exists to prevent that. It drops every k -mode with $l \geq L_{max_out}$, drops the corresponding (l, m) pairs, and rebuilds `lm_idx_per_mode` through a remap table so the surviving u indices are contiguous again.

```
k_mask = lm_l_np < self.L_max_out # keep modes with l < L_max_out
parent_j = parent_j[k_mask]
lm_idx_old = np.array(lm_idx_per_mode)[k_mask]

pair_mask = lm_pairs_np[:, 0] < self.L_max_out
lm_pairs = lm_pairs[pair_mask]
remap = np.full(len(pair_mask), -1, dtype=np.int32)
remap[np.where(pair_mask)[0]] = np.arange(int(pair_mask.sum()), dtype=np.int32)
lm_idx_per_mode = jnp.array(remap[lm_idx_old], dtype=jnp.int32)
```

The `remap` array is `-1` everywhere a pair was dropped. Because `k_mask` and `pair_mask` use the same threshold, no surviving k -mode can point at a dropped pair, so no `-1` ever reaches `lm_idx_per_mode`. This is an invariant worth asserting explicitly on a rewrite.

After truncation ψ genuinely has bandwidth L_{max_out} , but $\rho = |\psi|^2$ still has bandwidth $2 \cdot L_{max_out} - 1$. The forward SHT is performed at bandwidth L_{max_out} , so **ρ_{lm} is aliased**, not merely truncated. Power from $l \in [L_{max_out}, 2L_{max_out}-2]$ folds back into the retained modes. `Running_sims_L_reduc.py` exists to measure exactly how much this matters by sweeping L_{out_frac} and comparing against $L_{out_frac} = 1$. Do not treat truncation as free.

2.5 L-sharding alignment

Immediately after the truncation, `L_max_out` is rounded **down** to a multiple of the device count:

```
if self.sharding.shard_l is not None:
    n_dev = len(self.sharding.devices)
    if self.L_max_out % n_dev != 0:
        L_aligned = (self.L_max_out // n_dev) * n_dev
        self.L_max_out = L_aligned
```

This is because `ShardingManager.shard_l_arr` uses `jax.device_put`, which requires the sharded axis to divide exactly. If it does not, the method silently returns the array *replicated* — every device holds a full copy — which at these sizes means an immediate out-of-memory. Rounding `L_max_out` down guarantees the shard succeeds. Note this happens after `L_out_frac` is applied, so the effective bandwidth is slightly below what you asked for; the log line reports it.

The natural $2L - 1$ is always odd, so when `L_out_frac = 1` on 4 GPUs this rounding always fires and drops one to three modes.

Part III — Setup: `__init__` and `initialising_simulation`

3.1 `StellarSimTDep.__init__`

Pure bookkeeping — no arrays are built. It records the knobs, resolves the compute dtype pair, constructs the `ShardingManager`, and derives `dt` from `total_evolve_time / no_time_steps` (later overwritten if `dt_override` is set).

Parameter	Typical	Role
<code>m22</code>	10 – 50	Boson mass in units of 10^{-22} eV. Sets the whole problem size: <code>Nj</code> , <code>l_max</code> , and hence <code>Nmodes_k</code> , all grow steeply with it.
<code>SphHT</code>	<code>True</code>	Selects the spherical-transform density path. <code>False</code> selects the Gaunt-coefficient path (Section 5.4), which is only tractable at small <code>L</code> .
<code>L_out_frac</code>	0.1 – 1.0	Bandwidth truncation factor (Section 2.4).
<code>compute_dtype</code>	<code>complex64</code>	Dtype for the heavy density path. Energies and phases stay <code>float64/complex128</code> regardless.
<code>sparse_k_batch</code>	2^{18}	k-modes per block in the sparse matmul scan (Section 5.2).
<code>r_chunk_size</code>	32	Radial bins per chunk in the SHT round trip. Must divide the device count. Caps the <code>(r_chunk, L, 2L-1)</code> transient.
<code>l_band_size</code>	32 – 128	(l,m) modes per chunk in the Poisson solve (Section 6.2).
<code>dt_override</code>	e.g. 500	Safety divisor on <code>min(T_orb, T_c)</code> .
<code>ramp_time</code>	Gyr	Duration of the linear switch-on of the fluctuating field.
<code>no_radius_bins</code>	1000	<code>Nr</code> .
<code>use_multi_gpu</code>	<code>True</code>	Enables <code>Nj</code> - and <code>L</code> -axis sharding.

The dtype pair is derived, not passed:

```
self.compute_dtype = jnp.dtype(compute_dtype)
if self.compute_dtype == jnp.complex64:
    self.compute_real_dtype = jnp.float32
elif self.compute_dtype == jnp.complex128:
    self.compute_real_dtype = jnp.float64
else:
    raise ValueError(...)
```

and there is one eager validation: `r_chunk_size % n_dev == 0`, because the SHT round trip uses `shard_map` to split each radial chunk across devices.

3.2 `initialising_simulation` — the ten stages

This one method is ~470 lines and does everything up to the first timestep. It is best understood as ten sequential stages. On a rewrite, each of these should be its own function.

1. **Load or build the eigenstate library.** Cached to `precomputed_wf/*.npz + *.pkl`.
2. **Fix the bandwidth.** Compute `L`, apply `L_out_frac`, align to device count.
3. **Cast and shard `R_j_r`.** Pad the `Nj` axis to a multiple of the device count.
4. **Build the mode tables.** `precompute_lm_pairs`, then the `l ≥ L_max_out` cut.
5. **Draw the amplitudes.** `aj[k] = sqrt(aj_2[parent_j[k]]) · exp(i·U[0,2π))`.
6. **Pre-sort the modes for the sparse kernel.** `precompute_bin_blocks + _sort_modes_for_sphht`.
7. **Build the static density.** `compute_diagonal_rho_expansion → rho_static_r_l00, weight_j`.

8. **Place the stars.** Random directions at radius $\sim r_{\text{half}}$, circular velocities from M_{enc} .
9. **Wire up rebound.** Create the `Simulation`, register `additional_forces_step`.
10. **Choose `dt`.** From $\min(T_{\text{orb}}, T_{\text{c}}) / dt_{\text{override}}$, and recompute `no_time_steps`.

Stage 1 — the eigenstate library and its cache

Building the library is expensive, so it is cached under a key derived from the physical parameters that determine it:

```
cache_params = {
    'm22': float(self.m22),
    'r_min': float(self.r_min),
    'r_max_enclosing_frac': float(self.r_max_enclosing_frac),
    'no_radius_bins': int(self.no_radius_bins),
}
```

Two files are written. The `.npz` holds the bulk numeric arrays — `R_j_r` (as float32, "v2" format), `l`, `E`, `aj_2`, `total_mass`, `rmin`, `rmax` — and the `.pkl` holds the `jaxsp` objects. Only one thing is kept from the pickle: `eigenstate_lib.radial_eigenmode_params`, the pytree needed to evaluate `R_j(r)` at arbitrary radii every timestep. The rest is dropped immediately with `del objs` because the pickle is ~ 4 GB.

A "v1" fallback reads the same quantities out of the pickle when the `.npz` predates the `l` key. `_cache_valid` compares each stored parameter with `np.isclose` for floats and `!=` otherwise, and silently recomputes on mismatch.

The evaluator itself is a doubly-vmapped `jaxsp` call, defined once for all paths:

```
self._eval_library = jax.vmap(
    jax.vmap(jsp.eval_radial_eigenmode, in_axes=(None, 0)), # over j
    in_axes=(0, None)) # over r
```

so `self._eval_library(r_array, params)` returns `(len(r_array), Nj)`.

When the cache misses, the library is built from a hard-coded core-NFW-with-tides parameter vector (a specific halo fit). `rmax` is the radius enclosing `r_max_enclosing_frac` of the mass; `rmin` for the *potential* solve is `0.1 pc` but is then reassigned to `self.r_min * u.from_pc` for the *wavefunction* solve. The reassignment is easy to miss and matters, because `self.rmin` is what defines the radial grid.

Stage 3 — dtype, padding, sharding of `R_j_r`

`R_j_r` is the largest standing array: `(Nr, Nj)`, which at `m22 = 50` is `(1000, 717065)` — 2.9 GB in float32, 5.7 GB in float64. It is cast down and sharded along `Nj`. Sharding by `device_put` needs exact divisibility, so the `Nj` axis is padded first:

```
self.nj_pad = 0
if self.sharding.shard_nj is not None:
    n_dev = len(self.sharding.devices)
    pad = (-R_j_r_cast.shape[1]) % n_dev
    if pad:
        R_j_r_cast = jnp.pad(R_j_r_cast, ((0, 0), (0, pad)))
        self.l = jnp.pad(self.l, (0, pad))
        self.eigen_energies = jnp.pad(self.eigen_energies, (0, pad))
        self.nj_pad = int(pad)
```

Every per-`j` array must be padded identically or the contractions misalign. The padded slots are inert: no `parent_j[k]` ever points at them, so their `R` values (zero) and energies (zero) never enter a sum. `self.nj_pad` is remembered so that `R_j_at_particles` — which is evaluated fresh every timestep from `radial_eigenmode_params`, and therefore comes back *unpadded* — can be padded to match at the call

site inside `calc_rho_lm_at_parts_and_call_insert`.

This is a latent trap. `self.l` gets padded with zeros, so padded slots claim $l = 0$. `weight_j = aj_sq_j \cdot (2l+1)/4\pi` would give them weight $1/4\pi$ — except that `aj_sq_j` is zero there, because it is built by scattering only into `parent_j`. The two zeros cancel the mistake. If you ever change `weight_j` to not multiply by `aj_sq_j` first, the padding will silently inject fake monopole mass.

Stage 5 — the amplitudes

```
Nmodes = len(parent_j)
rand_phase_per_mode = jax.random.uniform(
    jax.random.PRNGKey(42), shape=(Nmodes,), minval=0.0, maxval=2*jnp.pi)
aj = (jnp.sqrt(aj_2[parent_j]) * jnp.exp(1j * rand_phase_per_mode)
     ).astype(self.compute_dtype)
```

A fixed key means the halo realisation is reproducible across runs — but note that `Nmodes` depends on `L_out_frac` through the mode cut, and `jax.random.uniform` produces a different sequence for a different shape. **Two runs with different `L_out_frac` therefore have different random phases**, not the same phases truncated. For the convergence study in `Running_sims_L_reduc.py` this means the comparison conflates truncation error with a different halo realisation. On a rewrite, draw the phases at full `Nmodes` *before* the cut and then index them with `k_mask`.

Stage 8 — initial conditions

Each star `i` gets a radius drawn uniformly from $[r_{\text{half}} - r_{\text{half_width}}/2, r_{\text{half}} + r_{\text{half_width}}/2]$, and an isotropic random direction built from three standard normals normalised to the unit sphere. The velocity is circular, of magnitude $\sqrt{G M_{\text{enc}}(r) / r}$ with `M_enc` interpolated from the static profile, and its direction is a random rotation within the plane perpendicular to $\hat{r}$:

```
ref = jnp.where(jnp.abs(r_i_unit[2]) < 0.9,
               jnp.array([0., 0., 1.]), jnp.array([1., 0., 0.]))
o_i_unit = jnp.cross(r_i_unit, ref); o_i_unit /= jnp.linalg.norm(o_i_unit)
t_i_unit = jnp.cross(r_i_unit, o_i_unit)
b_i_unit = jnp.cross(t_i_unit, r_i_unit)
v_i_unit = t_i_unit * jnp.sin(rand_theta) + b_i_unit * jnp.cos(rand_theta)
```

The `ref` switch avoids the degeneracy where $\hat{r}$ is parallel to the reference axis and the cross product vanishes. $\hat{t}$ and $\hat{b}$ then span the tangent plane and `rand_theta` picks a uniformly random direction in it — so the stars are on circular orbits with randomly oriented angular momenta.

Each star gets a `Simulation_Particle` (for history) *and* a rebound particle with `m = 0.0` (for integration). The two are kept in sync by hand in `time_step_particle`.

Stage 9 — the force callback

```
def additional_forces_step(_reb_sim):
    N = self.no_of_particles
    xyz = np.empty((N, 3))
    for i in range(N):
        p = sim_particles[i]
        xyz[i, 0], xyz[i, 1], xyz[i, 2] = p.x, p.y, p.z
    x, y, z = xyz[:, 0], xyz[:, 1], xyz[:, 2]
    r, theta, phi = SSF.Cartesian_to_sph_np(x, y, z)
    positions_sph = jnp.asarray(np.stack([r, theta, phi], axis=1))
    self._force_call_count += 1
    a_r_all, a_theta_all, a_phi_all = self.construct_acc_master_func(positions_sph)
    a_x, a_y, a_z = SSF.acceleration_spherical_to_cartesian_np(
        np.asarray(a_r_all), np.asarray(a_theta_all), np.asarray(a_phi_all), theta, phi)
    for i in range(N):
        sim_particles[i].ax += float(a_x[i])
        sim_particles[i].ay += float(a_y[i])
        sim_particles[i].az += float(a_z[i])
```

The key design decision: **all stars are done in one batched JAX call**, not one call per star. IAS15 invokes this callback several times per macro timestep — it uses seven Gauss-Radau nodes and iterates the predictor-corrector to convergence, so the count varies from step to step — and each invocation runs the entire density+Poisson pipeline for all stars at once. The `+=` on `ax/ay/az` is required — rebound may have already written other force contributions into those slots.

The coordinate transforms are done in numpy, batched, deliberately: doing them in JAX would dispatch `N+1` tiny kernels and stall on the host round trip anyway.

Stage 10 — the timestep

```
orbital_P = 2 * jnp.pi * r_orbits / jnp.array(init_vels)
min_orbital_P = jnp.min(orbital_P)
alive_j = jnp.unique(self.parent_j) # eigenstates surviving the l-cut
min_psi_t = jnp.min(-1 / self.eigen_energies[alive_j])
new_dt = min(min_orbital_P / self.dt_override, min_psi_t / self.dt_override)
self.sim.dt = float(new_dt); self.dt = new_dt
self.no_time_steps = int(self.total_evolve_time * self.u.from_Gyr / new_dt)
```

Note `alive_j`: after the `l ≥ L_max_out` cut, whole eigenstates may have vanished from ψ . Using `eigen_energies` over *all* `j` would pick up an energy belonging to a mode that no longer exists and give an unnecessarily small `dt`. This is a genuinely subtle detail and correct as written.

`no_time_steps` is recomputed here, and then **mutated a second time** in `run_simulation` to add the ramp steps. The final value is only known after that.

Part IV — Simulation_Particle

A plain Python/numpy bookkeeping object. It holds no JAX arrays and participates in no JIT. Its only job is to accumulate per-star history so that the analysis scripts can read it back.

Method	What it does
<code>__init__</code>	Stores the initial Cartesian state, converts to spherical, seeds every history list with its $t=0$ value. <code>kinetic_energy</code> starts at $\frac{1}{2} v ^2$ (per unit mass — the stars are massless test particles, so all energies are specific).
<code>Change_to_new_vel</code>	Overwrites the velocity and resets the velocity/KE/angular-momentum histories to a single element. Only used by the commented-out <code>v_circ_true</code> correction in <code>run_simulation</code> ; currently dead code.
<code>Create_V_array</code>	Preallocates <code>velocities_arr</code> of shape <code>(no_time_steps+1, 3)</code> with row 0 holding the initial spherical velocity. Must be called <i>after</i> <code>no_time_steps</code> is final.
<code>update_state</code>	Called once per macro timestep. Writes the new Cartesian and spherical state, appends to every history list, and recomputes the running velocity dispersion.

4.1 The velocity-dispersion definition

```
self.velocities_arr[self.time_step + 1] = self.v_sph
valid = self.velocities_arr[:self.time_step + 2]
new_vel_disp = (np.std(valid[:, 0])**2
                + np.std(valid[:, 1])**2
                + np.std(valid[:, 2])**2) ** 0.5
```

This is the *time*-dispersion of a single star's own spherical velocity components over its history so far — not the dispersion across the star population at a fixed time. It grows as the star is heated. Note it is computed over the whole history from $t=0$, so it includes the ramp phase and is a cumulative, not instantaneous, measure. Be careful interpreting it.

The `velocities` and `velocities_cart` Python lists duplicate `velocities_arr`, grow without bound, and are pickled into every checkpoint. At 12 231 steps $\times$ 100 stars this is the dominant checkpoint size. Drop them on a rewrite; `velocities_arr` already has everything.

Part V — Building the density field

This is the computational heart. The goal is `rho_lms` of shape `(Nr, L_max_out, 2·L_max_out - 1)`: the spherical-harmonic coefficients of p at every radial grid bin, at the current time. Plus the same quantity evaluated at each star's exact radius.

5.1 Call graph

```
Build_rho_lms_for_timestep(t)                                # once per macro timestep
└─ phase = exp(-i E_j t dt)                                # (Nj,) complex128
└─ SphHT → MSS.build_sphht_rho_lms_jit(...)                 # (Nr, L, 2L-1)
    └─ lax.scan over r-chunks:
        └─ sparse_a_u_j_matmul → S (N_unique, r_chunk)      == psi_lm(r)
        └─ scatter S into flm → (r_chunk, L, 2L-1)
        └─ s2fft.inverse → psi(r, θ, φ)
        └─ |psi|^2 * M_tot → rho(r, θ, φ)
        └─ s2fft.forward → rho_lm(r)
        └─ ramp blend + static monopole add
└─ else → construct_rho_lms_gaunt(...)                       # Gaunt path, small L only

calc_rho_lm_at_parts_and_call_insert(...)                   # many times per macro step (IAS15)
└─ _eval_library(r_p) → R_j_at_parts (Np, Nj)
└─ compute_rho_lm_at_particles_sphht(...)                   # same kernel, M = Np
└─ compute_rho_lm_at_particles_diagonal_only()               # static baseline, no SHT
└─ ramp blend
└─ lax.map over stars: insert_particle_rho_lm_and_get_philm
    └─ PS.compute_phi_lm_and_deriv(...)                     # the Poisson solve
```

5.2 `MSS.sparse_a_u_j_matmul` — the hot kernel

Computes $S[u, m] = \sum_{k : \text{lm_idx}[k] = u} a_k \cdot \text{phase}[\text{parent_j}[k]] \cdot R[m, \text{parent_j}[k]]$, i.e. a segmented sum over k -modes grouped by their (l, m) bin. Three requirements pull against each other:

1. **Bit-reproducibility.** The obvious implementation is a scatter-add, `S.at[u_k].add(...)`. On GPU that lowers to atomics, whose accumulation order varies between kernel launches. At complex64 this seeds $\sim 1e-5$ of run-to-run noise per step, which the chaotic stellar orbits amplify to $O(1)$ by late times. `jax.ops.segment_sum` does not help — even with `indices_are_sorted=True` it still lowers to atomics, and on this jaxlib it segfaults for complex inputs.
2. **Bounded memory.** The intermediate `contrib[m, k] = a_k · phase · R[m, parent_j[k]]` has shape `(M, Nmodes_k)`. At `m22=50` with `Nmodes_k = 58 878 110` and `M = 100` stars that is **47 GB**. It cannot be materialised.
3. **Compile-time sanity.** Any operation on a compile-time-constant array of 58.9M elements will be constant-folded by XLA on the host, single-threaded.

The resolution is to do the grouping **once, on the host, at setup**, and then scan over bin-aligned blocks at run time.

Step 1 — `precompute_bin_blocks` (host, numpy, once)

Sorts the k -modes by their bin index and partitions the sorted axis into contiguous blocks. The critical property: a block holds only **whole bins**. No (l, m) bin ever straddles a block boundary.


```

perm = np.argsort(lm_idx, kind='stable')
counts = np.bincount(lm_idx[perm], minlength=N_unique)
if counts.max() > k_block:
    raise ValueError(...) # a block must be able to hold the biggest bin
starts = np.concatenate([[0], np.cumsum(counts)])

spans = []; u0 = 0 # greedy pack whole bins into blocks
while u0 < N_unique:
    u1 = u0; base = starts[u0]
    while (u1 < N_unique
           and starts[u1+1] - base <= k_block
           and u1 - u0 < max_bins_per_block):
        u1 += 1
    spans.append((u0, u1)); u0 = u1

```

It returns a `BinBlocks` namedtuple: `perm` (the permutation, applied on the host), plus four device arrays describing the partition — `block_start` (first sorted-mode index of each block), and `bin_lo` / `bin_hi` / `bin_id` of shape `(n_blocks, B)` giving, for each bin in each block, its start and end offset *local to the block* and its destination row. Padding entries have `lo == hi == 0` and `bin_id = N_unique`, a dead row that is discarded.

Step 2 — `_sort_modes_for_sphht (host, once)`

```

perm = self.bin_blocks.perm
K = self.bin_blocks.k_block
aj_sorted = jnp.concatenate([aj[perm], jnp.zeros(K, aj.dtype)])
parent_j_sorted = jnp.concatenate([self.parent_j[perm], jnp.zeros(K, self.parent_j.dtype)])

```

The tail pad of `K` zeros lets the kernel take a fixed-width `dynamic_slice` for the final block without its start index being clamped. Padded amplitudes are zero, so the column of `R` their `parent_j` gathers is irrelevant.

`self.aj` and `self.parent_j` are deliberately **left in original k-order**. The Gaunt path pairs them positionally with `lm_idx_sorted_per_mode` and `lm_l_per_mode`, which are not permuted. Only the `_sorted` copies go to the SphHT kernel. Keep the two orderings separate or you will corrupt one path while fixing the other.

Step 3 — the run-time scan

```

if aj_phase is None: # callers in an r-chunk loop pass this in pre-folded
    aj_phase = fold_phase_into_aj(aj_sorted, parent_j_sorted, phase_c, R.dtype)
zeros_col = jnp.zeros((M, 1), cdtype)

def block_body(S, blk):
    st, lo, hi, bid = blk
    a_b = jax.lax.dynamic_slice(aj_phase, (st,), (k_batch,)) # (K,)
    p_b = jax.lax.dynamic_slice(parent_j_sorted, (st,), (k_batch,)) # (K,)
    contrib = a_b[None, :] * R[:, p_b] # (M, K)
    cs = jnp.cumsum(contrib, axis=1)
    cs = jnp.concatenate([zeros_col, cs], axis=1) # (M, K+1)
    sums = cs[:, hi] - cs[:, lo] # (M, B)
    return S.at[bid, :].add(sums.T), None

S0 = jnp.zeros((N_unique + 1, M), cdtype) # +1 = dead row for padded bins
S, _ = jax.lax.scan(block_body, S0, blocks)
return S[:N_unique]

```

Within a block, a prefix sum has a *fixed* reduction order, so it is bit-reproducible. Differencing the prefix sum at `lo` and `hi` extracts each bin's total. Because blocks are bin-aligned, every real bin is written exactly once across the entire scan, so the `.at[].add()` never has two live contributions racing for the same row — only the dead row `N_unique` sees collisions, and it is discarded. All three requirements are

satisfied simultaneously.

Accuracy bonus. An earlier version took the cumsum over the *entire* 58.9M-mode axis and differenced it at bin boundaries. That recovers a small bin sum by subtracting two enormous prefix sums — catastrophic cancellation at complex64. The blocked form differences over at most `k_batch` terms and is strictly better conditioned.

Hoisting `aj_phase` out of the `r-chunk` loop

`aj_phase` folds the per-`j` phase into the amplitudes. It depends only on `aj_sorted`, `parent_j_sorted`, `phase_c` and `cdtype` — **never** on `R`, only on `R`'s dtype. But `sparse_a_u_j_matmul` is called once per `r-chunk` from the density builders, so computing it inside the kernel recomputed a random gather over all `Nmodes_k` on every chunk: at `m22 = 50` that is a 201M-element gather, ~2.4 GB of traffic, redone `Nr / r_chunk` times per timestep. XLA does **not** reliably hoist an intermediate that large out of a `lax.scan` body, so it must be done by hand.

`MSS.fold_phase_into_aj(aj, parent_j, phase_c, R_dtype)` computes it once; the builders call it before their chunk loop and pass the result via the optional `aj_phase=` argument. The `aj_phase=None` default preserves the old behaviour for the single-shot particle-path call, which has no loop to hoist out of.

This is bit-identical, and so is `r_chunk_size`. Both facts follow from the same observation: `cdtype` is derived from exactly the same three dtypes either way, and the cumsum reduces along the `k` axis while `M` (= `r_chunk` in the density path) is only the batch axis. Changing `M`, or reusing a pre-folded `aj_phase`, reorders no reduction. Neither knob can perturb the determinism contract of Section 8.5 — which means both can be validated against an existing checkpoint bit-for-bit.

Choosing `sparse_k_batch`

The hard floor is the largest bin, `max_l n_l` — 1143 at `m22 = 50`, where `n_l` is the number of radial states with that `l`. Above the floor it is a pure trade between kernel-launch overhead (many small blocks) and the `(M, k_batch)` transient.

<code>sparse_k_batch</code>	Scan blocks (m22=50)	Peak transient, M=100 stars (contrib+cumsum+gather)
16 384	3 711	0.03 GiB
262 144 (recommended)	226	0.49 GiB
524 288	113	0.98 GiB
1 048 576	57	1.95 GiB

The transient is **replicated on every device**, not sharded: the `(M, k_batch)` gather and its cumsum live on each GPU. The floor is `m22`-dependent, so a value tuned at `m22 = 10` will be rejected at `m22 = 50` — `precompute_bin_blocks` raises with the exact minimum required, which is the right behaviour: it fails at setup rather than at hour three.

5.3 The SphHT round trip

Given $S[u, r] = \psi_{lm}(r)$, getting $p_{lm}(r)$ requires squaring, which is a convolution in harmonic space and a pointwise product in real space. So:

1. Scatter `S` into a dense `(r_chunk, L_out, 2L_out-1)` coefficient buffer `flm` at the positions `[lm_pairs[:,0], (L_out-1) + lm_pairs[:,1]]`. The `L_out-1` offset centres `m=0`.
2. `s2fft.inverse(flm, L_out, sampling='mw') → ψ` on the MW angular grid, `(n_theta, n_phi) = (L_out, 2L_out-1)`.

3. `rho = total_mass * |psi|2` — pointwise, trivial.
4. `s2fft.forward(rho, L_out, ...) → p_lm(r)`.

The transient `(Nr, L_out, 2L_out-1)` complex is what forces `r_chunk_size`: at `m22=10` with `c64` it is 3.7 GB, plus `s2fft`'s own working set. Streaming `r_chunk` radial bins at a time bounds it to `(r_chunk, L_out, 2L_out-1)`.

`build_sphht_rho_lms_jit` additionally folds the ramp blend and the static-monopole add *into the same scan*, per chunk, so that no second full-size `(Nr, L, 2L-1)` array is ever alive. And it writes into a pre-allocated, pre-sharded accumulator with `dynamic_update_slice` rather than using `lax.map`, whose stacked output would be allocated replicated on every device and defeat the sharding entirely.

```
# inside the chunk body, when out_sharding is given
S_T = lax.with_sharding_constraint(S_T, NamedSharding(mesh, P('x', None)))
rho_lms_chunk = shard_map(_sht_roundtrip, mesh=mesh,
                          in_specs=P('x', None),
                          out_specs=P('x', None, None),
                          check_rep=False)(S_T)
rho_lms_chunk = lax.with_sharding_constraint(rho_lms_chunk, out_sharding)
```

The `shard_map` splits the chunk along its **radial** axis (hence `r_chunk_size % n_dev == 0`), runs an independent SHT round trip per device, and the result is then constrained back to `L`-sharding for the accumulator.

5.4 The Gaunt path (`SphHT = False`)

The alternative to the round trip is to do the square directly in harmonic space, using Gaunt coefficients — the integrals of triple products of spherical harmonics:

$$\rho_{LM} \propto \sum_{\{lm\}} \sum_{\{l'm'\}} \psi_{lm} \cdot \psi_{l'm'}^* \cdot G(lm, l'm', LM)$$

This requires a precomputed sparse table of nonzero Gaunt coefficients, built once by `gf.precompute_gaunt_table`, and the `dense a_u_j` matrix of shape `(N_unique, Nj)`. The number of nonzero Gaunt triples grows roughly as L^5 . It is exact (no aliasing) and it is the right choice at small L ; it is completely intractable at $L = 240$.

When `SphHT = True`, `run_simulation` still has to supply the Gaunt arguments because they appear in the JIT signature, so it plants harmless placeholders:

```
self.a_u_j = jnp.zeros((1, 1), dtype=self.aj.dtype)
self._jit_all_i = jnp.zeros(1, dtype=jnp.int32)
self._jit_all_j = jnp.zeros(1, dtype=jnp.int32)
self._jit_all_G = jnp.zeros(1, dtype=jnp.float64)
self._jit_all_Lf = jnp.zeros(1, dtype=jnp.int32)
```

On a rewrite: pick one path. The dual-path structure means every function carries dead arguments, `self.aj/self.parent_j` must be kept in an ordering the SphHT path does not use, and the JIT signature is polluted with five placeholder arrays. If you need Gaunt for validation at small L , make it a separate module with its own entry point and compare offline.

5.5 The ramp

```
def ramp_frac_for_step(self, time_step):
    if time_step < self.n_ramp_steps:
        return (time_step + 1) / self.n_ramp_steps
    return 1.0
```

and the blend, applied identically on the grid and at the stars:

$$\rho = \rho_{\text{static}} + \text{ramp_frac} \cdot (\rho_{\text{full}}(t) - \rho_{\text{static}})$$

At `ramp_frac = 0` the stars see the exact smooth profile their circular velocities were built from, so they start on genuinely closed orbits. At `ramp_frac = 1` they see the full granular field. The ramp linearly switches on **all** of the time-dependent structure at once: the monopole's breathing *and* the $l \geq 1$ asphericity.

On the grid this is folded into the SHT scan. For the Gaunt path and at the stars it is done explicitly. Note the static piece only touches the ($l=0$, $m=0$) slot, which lives at index `[:, 0, L_out - 1]` because $m = 0$ sits at the centre of the $2L-1$ axis:

```
rho_lms = ramp_c * rho_lms_full
rho_lms = rho_lms.at[:, 0, L_out - 1].add(
    ((1.0 - ramp_c) * self.rho_static_r_l00).astype(rho_lms.dtype))
```

`ramp_c` is cast to `compute_real_dtype` first. Multiplying a `complex64` array by a `float64` scalar would promote the whole thing to `complex128` and break the sharded-output dtype contract downstream. This kind of accidental promotion is the single most common way to silently double memory in JAX.

Part VI — The per-star Poisson solve

6.1 `insert_particle_rho_lm_and_get_phi_lm` — grid refinement, not mass insertion

The star sits at radius `r_p`, generally between two grid bins. The multipole integrals are split at exactly `r_p`. Rather than interpolate, the code builds a virtual $(N_r + 1)$ -point grid with `r_p` spliced in at its sorted position.

```
insert_idx = jnp.searchsorted(self.r, particle_r)

# for i < insert_idx -> take r[i]
# for i == insert_idx -> take particle_r
# for i > insert_idx -> take r[i-1] (shifted; the star occupies one slot)
gather_idx = jnp.clip(self.all_idx - (self.all_idx > insert_idx).astype(jnp.int32),
                      0, self.Nr - 1)
r_updated = jnp.where(self.all_idx == insert_idx, particle_r, self.r[gather_idx])
insert_mask = self.all_idx == insert_idx
```

The same `gather_idx` / `insert_mask` pair is then handed to the Poisson solver, which applies it lazily to `rho_lms` one (l, m) -band at a time. The full $(N_r+1, L, 2L-1)$ inserted array is **never materialised** — at $m22 \approx 5$ with $N_r=1000$ and $L_{\text{out}}=481$ that transient alone was ~ 3.7 GiB, *per star*, inside a `lax.map`.

The two integration masks:

```
mask_int = jnp.arange(self.Nr) < insert_idx          # intervals below r_p
mask_ext = jnp.arange(self.Nr) < (self.Nr - insert_idx) # intervals above r_p (reversed)
```

There are N_r trapezoid intervals between the N_r+1 points. `mask_int` selects the first `insert_idx` of them, which are exactly those lying below `r_p`. `mask_ext` selects the first $N_r - \text{insert_idx}$ intervals of the *reversed* array, i.e. those above `r_p`. Edge cases fall out correctly: a star outside the grid gives `insert_idx = Nr` and an empty exterior; inside gives `insert_idx = 0` and an empty interior.

6.2 `PS.compute_phi_lm_and_deriv`

Evaluates $\Phi_{lm}(r_p)$ and $d\Phi_{lm}/dr(r_p)$ for all $L_{\text{max_out}}^2$ modes. Two things make it non-trivial.

Ratio folding — the precision fix

Written naively, the interior integrand carries $r'^{(l+2)}$ and the prefactor $r_p^{-(l+1)}$. In Schrödinger units r is not $O(1)$, and at $l \sim 240$ these factors span hundreds of orders of magnitude. They overflow float64 individually even though their *product* is perfectly well behaved. The fix is to fold the prefactor into the integrand so every factor is a ratio ≤ 1 :

$$\begin{aligned} r_p^{-(l+1)} \cdot r'^{(l+2)} &\rightarrow r' \cdot (r' / r_p)^{(l+1)} && [\text{interior}, r' \leq r_p] \\ r_p^l \cdot r'^{(1-l)} &\rightarrow r' \cdot (r_p / r')^l && [\text{exterior}, r' \geq r_p] \end{aligned}$$

```
ratio_int = jnp.clip(r_updated / rp, 0.0, 1.0)
ratio_ext = jnp.clip(rp / r_updated, 0.0, 1.0)
r_fold_int = r_updated[None,:] * jnp.power(ratio_int[None,:], (ell_f+1.0)[:None])
r_fold_ext = r_updated[None,:] * jnp.power(ratio_ext[None,:], ell_f[:None])
```

The `clip` to 1 makes the out-of-region entries harmless — they are masked out of the integrals anyway, but without the clip they would be >1 raised to a large power and would overflow to `inf` before the mask ever ran. This was the dominant cause of spurious high- l forces and particle ejection at

higher `m22`.

Because the prefactor is now inside the integral, Φ and $d\Phi/dr$ share the *same two integrals* and differ only by per-`l` scalars:

```
prefix_arr      = -4.0 * jnp.pi * G / (2.0 * ell_f + 1.0)
dphi_ext_scale = ell_f / rp
dphi_int_scale = (ell_f + 1.0) / rp
...
dphi_band = pref * (ext_scale * integral_ext - int_scale * integral_int)
phi_band  = pref * (integral_ext + integral_int)
```

which is exactly the pair of formulas in Section 1.4.

l-banding — the launch-overhead fix

`L_max_out2` is 57 600 modes. Vmapping the per-mode gather over that would issue 57 600 tiny gathers. Instead the modes are chunked into bands of `l_band_size` and each band does exactly two bulk gathers:

```
rho_band      = rho_lms[:, l_vals, m_inds]          # (Nr, l_band_size)
f_orig_band   = rho_band[gather_idx].T             # (l_band_size, Nr+1)
f_ins_band    = rho_lm_at_particle[l_vals, m_inds]  # (l_band_size,)
f_at_lm_band  = jnp.where(insert_mask[None, :], f_ins_band[:, None], f_orig_band)
```

Because `output_lm_pairs` is built in `l`-sorted order, an `l`-coherent band gathers the *same row* of `r_fold_int` / `r_fold_ext` for every mode in it, and XLA collapses the gather to a broadcast.

The integrals themselves are masked trapezoid sums. The exterior one is done on the reversed array — `dr_rev = diff(r_updated[:, :-1])` is negative, hence the leading minus:

```
avg_int = 0.5 * (integrand_int[:, 1:] + integrand_int[:, :-1])
integral_int = jnp.sum(jnp.where(mask_int[None, :], avg_int * dr[None, :], zero_c), axis=1)

integrand_ext_rev = integrand_ext[:, ::-1]
avg_ext = 0.5 * (integrand_ext_rev[:, 1:] + integrand_ext_rev[:, :-1])
integral_ext = -jnp.sum(jnp.where(mask_ext[None, :], avg_ext * dr_rev[None, :], zero_c), axis=1)
```

The radial grid is **log-spaced**, and these are plain trapezoid sums in `r`, not in `log r`. Near `rmin` the bins are extremely fine and this is accurate; near `rmax` they are coarse. Whether that matters depends on where the density has support. If you rewrite, consider integrating in `log r` ($\int f dr = \int f \cdot r d(\ln r)$), which makes the quadrature uniform in the actual sample spacing.

6.3 `combine_acc` and the angular functions

Straight contraction of Section 1.5:

```
a_r      = jnp.sum(-dphi_lm_dr_T * Ylm_all, axis=0).real
a_theta = jnp.sum(-phi_lm_T * dY_dtheta / particle_r[None, :], axis=0).real
a_phi    = jnp.sum(-phi_lm_T * dY_dphi / (particle_r[None, :]
                                         * jnp.sin(particle_theta[None, :])), axis=0).real
```

`dY_dphi` is free: `dY_dphi = 1j * m_vals * Ylm_all`.

`MSS.compute_Ylm_and_dtheta_jit` deserves a note. The obvious call, `jax.scipy.special.sph_harm_y`, requires `(n, m,  $\theta$ ,  $\phi$ )` to share a flat shape. Feeding it `(Nmodes,)` mode indices and `(Np,)` angles forces `$\theta$` to be broadcast up to `(Nmodes,)` per star; internally it then allocates an associated-Legendre cube of shape `(n_max+1, n_max+1, Nmodes)`. At `L_max = 481` that is ~ 430 GiB per star. The fix is to call

`_gen_associated_legendre` directly with `cos θ` of shape `(Np,)` — a ~9 MiB table — then gather `legendre[m], l, p`, multiply by `exp(i|m|φp)`, and apply the `m < 0` sign correction. `dY/dθ` comes from a JVP with tangent 1 in `θ`, so one pass gives both.

Part VII — The main loop and checkpointing

7.1 `run_simulation`

Sequence, in order:

1. `initialising_simulation()` — everything in Part III. Returns `aj`.
2. Rebuild `aj_sorted` / `parent_j_sorted` from the returned `aj` (identical value, but do not rely on that).
3. Set `Nr, all_idx = arange(Nr+1)`.
4. Gaunt path only: build the Gaunt table and the dense `a_u_j`. Otherwise plant placeholders.
5. Build `output_lm_pairs` — every `(l, m)` with `l < L_max_out`, in `l`-sorted order. This ordering is what makes the `l`-banding in the Poisson solve efficient.
6. Compute `n_ramp_steps` and **add it to** `no_time_steps`.
7. Build `rho_lms` at step 0.
8. Compute the initial potential energy of every star (`construct_acc_master_func(..., poten=True)`).
9. `Create_V_array(no_time_steps)` — only valid now that `no_time_steps` is final.
10. Try `_load_checkpoint`; then loop.

```
while self.time_step < self.no_time_steps:
    self.rho_lms = None # free BEFORE building the replacement
    self.rho_lms = self.sharding.shard_l_arr(
        self.Build_rho_lms_for_timestep(self.time_step).astype(self.compute_dtype))

    self.time_step_particle() # rebound integrates one macro dt

    self.current_phase = jnp.exp(-1j * self.eigen_energies
                                   * (self.time_step + 1) * self.dt)
    r_pos_sphs_new = jnp.array([p.r_pos_sph for p in self.particles])
    _, _, _, phi_lm_new, Ylm_new = self.construct_acc_master_func(r_pos_sphs_new, poten=True)
    phi_at_parts = jnp.sum(phi_lm_new * Ylm_new.T, axis=1).real
    for i, particle in enumerate(self.particles):
        particle.potential_energy.append(float(phi_at_parts[i]))

    self.time_step += 1
    if self.time_step % checkpoint_every == 0:
        self._save_checkpoint(checkpoint_dir)
```

The `self.rho_lms = None` on the first line of the loop is not cosmetic. Without it, the right-hand side allocates the new `(Nr, L, 2L-1)` array while the old one is still referenced, doubling peak memory at the worst possible moment.

7.2 `time_step_particle`

`Rebound` and `Simulation_Particle` hold duplicate state and are synchronised by hand: write Python state into `rebound`, integrate to `sim.t + dt`, read it back.


```

for i, particle in enumerate(self.particles):
    p = self.sim_particles[i]
    p.x, p.y, p.z = map(float, particle.r_pos)
    p.vx, p.vy, p.vz = map(float, particle.v)

self._force_call_count = 0
self.sim.integrate(self.sim.t + self.dt)

for i, particle in enumerate(self.particles):
    p = self.sim_particles[i]
    particle.update_state([p.x, p.y, p.z], [p.vx, p.vy, p.vz])

```

During `sim.integrate`, IAS15 calls `additional_forces_step` many times at trial positions (`_force_call_count` reports how many). Each call runs the *entire* per-star pipeline: evaluate R_j at the trial radii, compute ρ_{lm} there, insert into the grid, solve Poisson for 57 600 modes, contract with Y_{lm} . The `_force_call_count` print exists to make that cost visible.

7.3 The frozen-phase approximation

`Build_rho_lms_for_timestep(t)` sets `self.current_phase` for time $t \cdot dt$. Every force evaluation inside the subsequent `sim.integrate` reads that same `self.current_phase`. The ULDM field therefore does not evolve *within* a macro timestep — it is a staircase in time, updated once per `dt`. IAS15 is an adaptive high-order integrator and is being fed a force that is discontinuous at every macro step boundary; its error estimator cannot see this. This is the reason `dt` must resolve the field coherence time T_c , not merely the orbital period.

The line after `time_step_particle` re-evaluates `current_phase` at $(t+1) \cdot dt$ purely so the potential-energy diagnostic is measured with the field at the star's new time. It is then immediately overwritten by the next iteration's `Build_rho_lms_for_timestep`.

7.4 Checkpointing

`_save_checkpoint` pickles per-star history plus `sim_time_step`, `no_time_steps`, `n_ramp_steps`, and `sim.t`. It does **not** save `rho_lms` (recomputed from `time_step`), the amplitudes `aj` (regenerated from the fixed PRNG key), or rebound's internal state — which is fine, because `time_step_particle` rewrites rebound's particle state from the Python objects at the top of every step.

`_load_checkpoint` prefers `checkpoint_final.pkl`, else the highest-numbered `checkpoint_step_N.pkl`. It is called *after* `Create_V_array`, so the preallocated `velocities_arr` is immediately replaced by the saved one.

- `maximum_rho_00` is **not** checkpointed and resets on resume, so a resumed run has a truncated diagnostic series.
- `no_time_steps` is restored *from the checkpoint*, overwriting the freshly computed value. If you change `total_evolve_time` and resume, the old value silently wins.
- IAS15 keeps internal predictor state across `integrate` calls, which is discarded on resume. A resumed run is not bit-identical to an uninterrupted one.

Part VIII — Memory and performance engineering

This part is why the file looks the way it does. Every item below was a real failure that had to be diagnosed from a job that either OOM'd or exceeded walltime.

8.1 The mixed-precision contract

Quantity	Dtype	Why
<code>eigen_energies</code>	float64	Multiplied by <code>t</code> at every step. Phase error compounds over 12 000 steps; float32 would drift badly.
<code>current_phase</code>	complex128	Result of <code>exp(-i E t)</code> . Cast down only at the point of contraction with <code>R_j</code> .
<code>R_j_r_fixed</code> , <code>R_j_at_parts</code>	float32	The (N_r, N_j) monster. Halves 5.7 GB $\rightarrow$ 2.9 GB at <code>m22=50</code> .
<code>aj</code> , <code>rho_lms</code> , <code>S</code>	complex64	The heavy density path.
<code>ramp_c</code>	float32	Cast explicitly. A float64 scalar $\times$ complex64 array promotes the whole array to complex128.
Gaunt coefficients	float64	Small, and precision-sensitive.

The rule: **energies and phases in double, everything that scales with `N_j` or `Nmodes` in single**. The cast points are all explicit, and every one of them is load-bearing.

8.2 Sharding

`ShardingManager` builds a 1-D device mesh with axis name `'x'` and exposes three placements:

Helper	PartitionSpec	Applies to
<code>shard_nj_arr</code>	<code>P(None, 'x')</code>	(N_r, N_j) and (N_{unique}, N_j) — split the <code>N_j</code> axis.
<code>shard_j_arr</code>	<code>P('x')</code>	1-D $(N_j,)$ arrays.
<code>shard_l_arr</code>	<code>P(None, 'x', None)</code>	$(N_r, L, 2L-1)$ — split the <code>L</code> axis.

`shard_l_arr` uses `jax.device_put`, which demands exact divisibility and **silently returns the array replicated** if the axis does not divide. Replicating a $(1000, 240, 479)$ complex64 array puts 920 MB on every device instead of 230 MB — and at full bandwidth it is far worse. This is why `L_max_out` is rounded down (Section 2.5). `jax.lax.with_sharding_constraint` *inside* a JIT has no such restriction (GSPMD pads internally), so the in-JIT path is strictly stronger. Prefer it.

`build_sparse_au_j` (Gaunt path) exists solely because `jnp.zeros((N_unique, N_j)).at[...].add(aj)` allocates the zeros *and* a fresh result array, peaking at $2\times$ the final size — ~ 26 GB at `m22=10` with complex64, which overflows a 25 GB GPU. Doing the scatter in numpy on the host and transferring once avoids the peak entirely. CPU RAM is cheap.

8.3 The bound-method JIT constant trap

This is the most important lesson in the file, and it cost a 24-hour job.

```
self.calc_rho_lm_at_parts_and_call_insert_jit = jax.jit(
    self.calc_rho_lm_at_parts_and_call_insert)    # <- BOUND method
```

Jitting a bound method captures `self` in the closure. Every array reached through `self` inside the traced function enters the jaxpr not as an argument but as a **literal constant**. XLA is then free to — and does — constant-fold operations on it.

The old `sparse_a_u_j_matmul` contained `order = jnp.argsort(lm_idx)`, where `lm_idx` came from `self.lm_idx_per_mode`. XLA saw `argsort(<constant of 58 878 110 int32>)` and evaluated it at compile time, on the host, single-threaded:

```
E0708 12:34:56 slow_operation_alarm.cc:140] The operation took 1h1m47.05135284s
Constant folding an instruction is taking > 1s:
  %sort.3 = (s32[58878110], s64[58878110]) sort(%constant.4601, %iota.68)
    op_name="jit(calc_rho_lm_at_parts_and_call_insert)/.../jit(argsort)/sort"

E0708 12:36:34 rendezvous.cc:116] [id=1] This thread has been waiting for
`Acquire clique: devices=4:[0,1,2,3]` for 10 seconds and may be stuck.
```

Rank 0 spent over an hour compiling; the other three ranks sat in the collective rendezvous waiting for it; the 24-hour walltime expired before a single timestep ran. Note the `%constant.4601` in the HLO — that is the tell.

Two independent fixes, both applied:

1. **Hoist the fixed permutation to setup.** `argsort` of a constant is a constant. It belongs in numpy, at startup, once. (`precompute_bin_blocks`.)
2. **Pass large arrays as traced arguments, not through `self`.** `compute_rho_lm_at_particles_sphht` now takes `sparse_a_u_j` explicitly, and `calc_rho_lm_at_parts_and_call_insert` forwards it. They stay device buffers instead of becoming ~1 GB of executable literals.

How to check, on any jitted function you suspect:

```
hlo = f.lower(*args).compile().as_text()
print('sort ops:', hlo.count(' sort('))
print('constants:', hlo.count('constant('))
```

The general rule for a rewrite: **never `jax.jit` a bound method that touches large `self` arrays.** Either make the class a registered pytree, or use `functools.partial(jax.jit, static_argnums=0)` with `__hash__/_eq__` defined and accept that `self` is then a compile-time constant *by design*, or — simplest and best — keep the numerics in free functions that take everything explicitly, and let the class be a thin orchestrator. The last option is what the module-level helpers in `Memory_speed_savers.py` already are.

8.4 The chunking knobs, summarised

Knob	Bounds the transient	Cost of lowering it
<code>sparse_k_batch</code>	(<code>M</code> , <code>k_batch</code>) in the sparse matmul	More scan blocks → more kernel launches. See the table in Section 5.2.
<code>r_chunk_size</code>	(<code>r_chunk</code> , <code>L</code> , <code>2L-1</code>) in the SHT round trip	More chunks; must divide <code>n_dev</code> for the <code>shard_map</code> ; and — the trap — it starves the sparse matmul . See below.
<code>l_band_size</code>	(<code>l_band</code> , <code>Nr+1</code>) in the Poisson solve	More bands; loses the <code>l</code> -coherent gather-to-broadcast collapse.
<code>L_out_frac</code>	Everything with an <code>L</code> axis	Physical aliasing of <code>p</code> , not just cost. Not a free knob.
<code>compute_dtype</code>	Everything	Precision.

`r_chunk_size` is **not** a pure memory knob, and setting it too low is expensive. `r_chunk` becomes `M`, the leading axis of the `(M, k_batch)` `contrib` tensor — and `M` is the *only* batch dimension the block cumsum has to parallelise over. At `r_chunk = 8` the cumsum runs over a 262 144-long axis with just 8 independent rows, which cannot fill an L40S's 142 SMs, and the scan runs $Nr/r_chunk \times n_blocks$ times (96 125 sequential iterations at `m22 = 50`). The total FLOP count is $M \times Nmodes_k$ either way, so a larger `r_chunk` buys occupancy for free. The memory it costs is slight: at `r_chunk = 128`, complex64, `m22 = 50` the `contrib` transient is 268 MB and the SHT working array 471 MB (r-sharded 4 ways → 118 MB/device) — against 48 GB/GPU. Going from 8 to 128 was worth several-fold on the density build. Keep it a multiple of `n_dev`.

8.5 Determinism

The chaotic amplification is the reason determinism is treated as a hard requirement rather than a nicety. Two runs of the same halo realisation must agree bit-for-bit, because a $1e-5$ discrepancy at complex64 in an early step becomes $O(1)$ in the stellar orbits by 10 Gyr. Every reduction over k-modes therefore has a fixed order:

- **Forbidden:** `.at[].add()` with colliding indices, `jax.ops.segment_sum` — both lower to GPU atomics.
- **Used instead:** `cumsum` within bin-aligned blocks, then difference at bin boundaries. Prefix sums have a fixed reduction order.
- `bincount` is fine — it is integer-valued, so accumulation order is irrelevant.
- A two-element floating-point sum *is* order-independent (addition is commutative); it is *associativity* that fails. This is why the padded dead row `N_unique` may collide freely.

Part IX — A guide to rewriting this

9.1 Suggested module structure

```

units.py      # Schroedinger units, constants
halo.py       # jaxsp wrapper: eigenstate library, caching, aj_2, total_mass
modes.py      # index bookkeeping: j/k/u tables, the l-cut, bin blocks
              # -> pure numpy, no JAX, fully unit-testable
field.py      # psi_lm (sparse matmul), rho_lm (SHT round trip), the ramp
              # -> free functions, all arrays passed explicitly
poisson.py    # phi_lm and dphi_lm/dr from rho_lm (multipole, ratio-folded)
angular.py    # Y_lm and dY_lm/dtheta
dynamics.py   # rebound wiring, force callback, the macro loop
state.py      # particle history, checkpointing
config.py     # one frozen dataclass of knobs, validated at construction

```

The single structural change that matters: `field.py` and `poisson.py` must contain no classes. Free functions taking explicit arrays. The orchestrator holds state; the numerics do not. That alone eliminates the entire class of bug in Section 8.3.

9.2 Build order, with a test at each stage

1. **Indices.** `modes.py`, in `numpy`. Test: $\sum_j (2 l_j + 1) == N_{\text{modes}_k}$; every `u` is hit by at least one `k`; `lm_pairs[lm_idx[k]] == (lm_l[k], lm_m[k])` for all `k`; after the `l-cut`, `lm_idx.max() == N_unique - 1` and no `-1` survives.
2. **Static density.** `p_static(r)` from `weight_j`. Test: $\int 4\pi r^2 \rho_{\text{static}} dr \approx \text{total_mass}$ to the accuracy of the radial grid. This is a strong end-to-end check on `aj_2`, the units, and `Enclosed_mass`.
3. **ψ_{lm} .** The sparse matmul. Test against a dense `a_u_j @ (phase * R.T)` on a small case (`Nj ~ 200`, `l_max ~ 10`). Then test bit-reproducibility: two identical calls must give `np.array_equal`, not `allclose`.
4. **ρ_{lm} .** The SHT round trip. Test: at `t = 0` with all random phases set to zero and only `l = 0` modes retained, `p_lm[:, 0, L-1] / sqrt(4π)` must equal a directly evaluated $M \cdot |\psi|^2$. Also test that `p_lm[:, 0, L-1]` is real to round-off (it is a real field's monopole).
5. **Poisson.** Test against an analytic case: a uniform sphere gives $\Phi(r) = -GM(3R^2 - r^2)/2R^3$ inside. Feed its `p_00` in and check $\Phi_{00} \cdot Y_{00}$ and $d\Phi/dr$. Then test the ratio folding by running at `l = 200` and confirming no `inf/nan`.
6. **Accelerations.** Test: for the spherically symmetric static field, `a_theta` and `a_phi` must vanish to round-off, and `a_r` must equal $-G M_{\text{enc}}(r)/r^2$. This catches sign errors, the `m`-offset in the `2L-1` axis, and `Y_lm` normalisation in one shot.
7. **Orbits.** Static field only, `ramp_frac == 0`. A star launched at `v_circ` must stay on a circle to the integrator tolerance for many orbits. Energy $\frac{1}{2}v^2 + \Phi$ must be conserved.
8. **Full field.** Only now switch the fluctuations on.

Test 6 is the highest-value test in the list. Almost every indexing mistake in this code — the `L_out - 1` centring of `m`, the `l`-sorted ordering of `output_lm_pairs`, the sign of $d\Phi/dr$, the conjugation convention in `Y_lm` — produces a nonzero `a_theta` on a spherically symmetric field. Write it first, run it constantly.

9.3 Traps, ranked

#	Trap	Symptom
1	JIT-ing a bound method with big <code>self</code> arrays (§8.3).	Compile takes hours; multi-GPU rendezvous timeouts; job dies at walltime with zero timesteps done.
2	Materialising (M, N_{modes_k}) or $(N_r+1, L, 2L-1)$.	OOM. <code>Failed to allocate 12.39GiB</code> .
3	float64 scalar $\times$ complex64 array.	Silent promotion to complex128; memory doubles; sharded-output dtype contract breaks downstream.
4	Scatter-add over k-modes (atomics).	Runs fine, results differ run to run at the $1e-5$ level, diverge to $O(1)$ by 10 Gyr. Nearly undetectable without an explicit <code>array_equal</code> test.
5	Unfolded $r^{(l+2)} / r^{-(l+1)}$ in the multipole integrals.	<code>inf/nan</code> in high- l modes; spurious forces; particles ejected. Worse at higher m_{22} .
6	<code>L_max_out</code> not divisible by <code>n_dev</code> .	<code>shard_l_arr</code> silently replicates; instant OOM.
7	Forgetting to pad <code>R_j_at_particles</code> by <code>nj_pad</code> .	Shape mismatch — this one at least fails loudly.
8	Drawing random phases <i>after</i> the l -cut.	Different <code>L_out_frac</code> gives a different halo realisation, so a convergence study measures the wrong thing.
9	Assuming <code>rho_lm</code> is truncated rather than aliased.	Systematic error in the retained modes that does not vanish as you refine anything else.
10	Trusting <code>no_time_steps</code> before <code>run_simulation</code> has added the ramp.	<code>velocities_arr</code> sized wrong; <code>IndexError</code> deep in <code>update_state</code> .

9.4 Things worth changing

- Rename `insert_particle_rho_lm_and_get_phi_lm`. Nothing is inserted; a grid node is refined. Call it `phi_at_star_radius`.
- Delete the Gaunt path from the main file, or delete the SphHT path. Carrying both pollutes every signature and forces two mode orderings to coexist.
- Draw the random phases before the l -cut and index them (§9.3 trap 8).
- Drop the `velocities` / `velocities_cart` Python lists; `velocities_arr` supersedes them and they dominate checkpoint size.
- Integrate the multipole integrals in `log r`, matching the log-spaced grid.
- Make the frozen-phase approximation explicit and configurable — at minimum, assert that `dt < T_c / N` for a stated N , and consider evaluating the phase at the actual IAS15 sub-step time rather than the macro-step time.
- Replace the `hasattr` JIT cache (`if not hasattr(self, '..._jit')`) with a module-level `@jax.jit` free function. It is the same trap as §8.3 wearing a hat.
- Checkpoint `maximum_rho_00`, and do not let a checkpoint overwrite a freshly computed `no_time_steps` without at least warning.

9.5 Shape reference

Symbol	Shape	Dtype	Notes
<code>R_j_r_fixed</code>	(N_r, N_j)	f32	Sharded on <code>N_j</code> . Padded to <code>n_dev</code> .
<code>R_j_at_parts</code>	(N_p, N_j)	f32	Rebuilt every force call. Pad by <code>nj_pad</code> .
<code>aj, aj_sorted</code>	$(N_k,), (N_k+K,)$	c64	<code>aj_sorted</code> is bin-sorted + tail-padded.

Symbol	Shape	Dtype	Notes
parent_j	(Nk,)	i32	$k \rightarrow j$.
lm_idx_per_mode	(Nk,)	i32	$k \rightarrow u$.
lm_pairs_jax	(N_unique, 2)	i32	$u \rightarrow (l, m)$.
eigen_energies	(Nj,)	f64	Padded.
current_phase	(Nj,)	c128	Cast to c64 at contraction.
S (= ψ_{lm})	(N_unique, M)	c64	$M = N_r$ chunk or N_p .
rho_lms	(Nr, L_out, 2L_out-1)	c64	Sharded on L_out.
rho_lm_at_particles	(Np, L_out, 2L_out-1)	c64	
output_lm_pairs	(L_out ² , 2)	i32	l-sorted. Order matters.
phi_lm, dphi_lm_dr	(Np, L_out ²)	c64	Output of the Poisson solve.
Ylm_all, dY_dtheta	(L_out ² , Np)	c128	Note the transposed layout.

9.6 Closing note

The physics in this file would fit on two pages. Everything else is a consequence of three facts: the mode count grows like $m^{22^4} - m^{22^5}$; JAX needs static shapes; and GPU atomics are not reproducible. If you keep those three in view, the rest of the design follows almost forcibly — sort once on the host, scan over fixed-width blocks, pass arrays explicitly, fold your prefactors into your integrands, and never let a float64 scalar touch a complex64 array.